From λ-Calculus to Extended SKI Combinator Calculus: Compilation and Call-by-Name Execution

Rodrigo Gomes de Araújo (202205515)

March 25, 2026

Contents

1 Introduction **1**

2 Background **1**

2.1 Classical Combinatory Logic 1

2.2 Extended Combinators 1

3 Fun **1**

3.1 Abstract Syntax 2

3.2 Parsing 2

4 Compilation **2**

4.1 Target Language 3

4.2 Direct Translation 3

4.3 Abstraction Elimination 3

4.4 Compilation Example 4

5 Evaluation **4**

5.1 Operational State 5

5.2 Combinator Reduction 5

5.3 Primitive Evaluation 5

5.4 Evaluation Example 6

6 Towards Graph Reduction **6**

6.1 Address-Based Representation 6

6.2 Heap Evaluation 6

7 Conclusion **7**

1 Introduction

This report was produced as an assignment for the *Implementation of Programming Languages* course. It presents the implementation of a small compiler pipeline for a functional language, from parsing to execution. The system reads source programs written in a custom `.fun` syntax, parses them into an abstract syntax tree, compiles them into an extended combinatory language based on SKI Calculus, and evaluates the resulting combinator expressions by reduction to weak head normal form. The implementation was written in Haskell. Its theoretical basis lies in the λ-Calculus, combinatory logic, and later refinements of bracket abstraction due to work of Church, Schönfinkel, and Turner [1, 2, 3].

2 Background

2.1 Classical Combinatory Logic

Combinatory logic was introduced by Schönfinkel as a formal system in which variable binding is eliminated and computation is expressed entirely through primitive operators [2]. Curry later developed the classical `S`-`K` basis and established the correspondence between combinatory logic and the λ-Calculus [5].

The standard abstraction rules are:

$$\lambda x.x = I \qquad \lambda x.P = K\,P \quad (x \notin fv(P)) \qquad \lambda x.(P\,Q) = S\,(\lambda x.P)\,(\lambda x.Q)$$

This translation is complete, but often produces large expressions because both sides of an application are abstracted independently.

2.2 Extended Combinators

To reduce this growth, Turner introduced additional combinators that preserve application structure during translation [3]:

$$B\,P\,Q\,R \to P\,(Q\,R) \qquad C\,P\,Q\,R \to (P\,R)\,Q$$

These combinators are specialized cases of `S`, allowing abstraction to focus only on the side of the application where the variable remains free.

3 Fun

`Fun` is a small functional language adapted from the course material. It extends the λ-Calculus core with integer constants, arithmetic operations, the conditional `IfZero`, local definitions through `Let`, and recursion through `Fix`. Although the original version presented in the lectures was evaluated under call-by-value, it was adapted to be compiled and evaluated to `WHNF` following the reduction strategy presented in the slides. Since no heap-based sharing is implemented, duplicated computations persist, making the evaluator operationally closer to call-by-name than true graph reduction.

3.1 Abstract Syntax

The source language is represented by the datatype `Term`:

```
data Term = Var Ident
          | Lambda Ident Term
          | App Term Term
          | Const Int
          | Term :+ Term
          | Term :- Term
          | Term :* Term
          | Term :/ Term
          | IfZero Term Term Term
          | Let Ident Term Term
          | Fix Term
```

Variables are represented by identifiers:

```
type Ident = String
```

The language consists only of expressions. Local definitions and recursion are expressed explicitly through `Let` and `Fix`.

3.2 Parsing

The front-end consists of a hand-written lexer and a parser generated with Happy. The lexer recognizes reserved words such as `let`, `in`, `if`, `iszero`, `then`, `else`, `fix`, and `lambda`, together with identifiers, integer literals, and punctuation symbols. The parser constructs values of type `Term`, supporting both `lambda x.e` and \`x.e` notation, enforcing left-associative application and the intended arithmetic precedence.

Programs are written as `.fun` files and read by the executable before being lexed, parsed, compiled, and evaluated.

4 Compilation

To execute `Fun` programs through combinator reduction, source terms must be translated into combinator expressions. This requires defining the target language and the rules that map source constructs into it.

4.1 Target Language

Compilation produces expressions of type `Comb`:

```
data Comb = Var Ident
          | Comb :@ Comb
          | S | K | I
          | B | C
          | Y
          | Const Int
          | Prim Op
          | IfZero
```

Applications are represented explicitly through `:@`, preserving their left-associative structure.

4.2 Direct Translation

The compiler is implemented by the function `compile :: Term -> Comb`. Constants, variables, and applications are translated homomorphically:

$$n = n \qquad x = x \qquad e_1\, e_2 = e_1 : @e_2$$

Arithmetic expressions are compiled into applications of primitive operators. For example:

$$e_1 + e_2 = (\texttt{Prim Add} : @e_1) : @e_2$$

The same scheme is used for subtraction, multiplication, and division.

Conditionals are compiled through the primitive combinator `IfZero`:

$$\texttt{IfZero}\ e_1\ e_2\ e_3 = ((\texttt{IfZero} : @e_1) : @e_2) : @e_3$$

Local definitions are eliminated before translation:

$$\texttt{Let}\ x\ e_1\ e_2 = (\lambda x.e_2)\, e_1$$

Recursion is compiled through the fixed-point combinator:

$$\texttt{Fix}\ e = Y : @e$$

4.3 Abstraction Elimination

λ-abstraction is removed by the function `lambdaT :: Ident -> Comb -> Comb`. The translation follows an improved SKI-style elimination using the extra combinators `B` and `C`, in the spirit of Turner's improved bracket abstraction algorithm [3].

The base cases are:

$$\lambda x.x = I \qquad \lambda x.P = K\,P \quad (x \notin fv(P))$$

A direct optimization is also applied when the bound variable appears only as the final argument:

$$\lambda x.(P\,x) = P \qquad (x \notin fv(P))$$

For applications, the compiler distinguishes three cases:

$$\begin{array}{ll} \lambda x.(P\,Q) = B\,P\,(\lambda_x^T.\,Q) & \text{if } x \notin fv(P) \\ \lambda x.(P\,Q) = C\,(\lambda_x^T.\,P)\,Q & \text{if } x \notin fv(Q) \\ \lambda x.(P\,Q) = S\,(\lambda_x^T.\,P)\,(\lambda_x^T.\,Q) & \text{otherwise} \end{array}$$

The choice is determined by free-variable analysis over combinator expressions using the function `fv`. Unsupported lambda shapes are rejected with a compile-time error, although all terms generated by the source-language compiler are intended to remain within the supported fragment.

4.4 Compilation Example

Consider the simple higher-order term:

$$\lambda x.\lambda y.\,y\,x$$

Internally, it is represented as:

```
Lambda "x" (Lambda "y" (App (Var "y") (Var "x")))
```

Compilation produces:

```
C :@ I
```

This follows directly from the abstraction rules: the inner abstraction places the free occurrence of x on only one side of the application, which allows the use of Turner's `C` combinator rather than the more general `S`.

5 Evaluation

Evaluation is performed directly over combinator expressions by weak reduction. The evaluator repeatedly exposes the head of the application spine, applies one local rewrite when possible, and stops when no head rewrite applies. This corresponds to weak reduction, since no rewriting is performed inside arguments unless required by primitive operators.

5.1 Operational State

The evaluator works over the types:

```
type Stack = [Comb]    type Spine = [Comb]
```

The auxiliary function `unwind` transforms nested applications into spine form by flattening left-associated applications:

$$((e_1 : @e_2) : xs) \mapsto \texttt{unwind}\ (e_1 : e_2 : xs)$$

This makes the head combinator and its available arguments explicit.

5.2 Combinator Reduction

The function `rewrite` performs one local combinator reduction step on a spine whenever enough arguments are available. The implemented rules are:

$$I\,x \to x \qquad K\,p\,q \to p \qquad S\,p\,q\,r \to (p\,r)\,(q\,r)$$

together with the extended combinators:

$$B\,p\,q\,r \to p\,(q\,r) \qquad C\,p\,q\,r \to (p\,r)\,q \qquad Y\,f \to f\,(Y\,f)$$

If no rewrite rule applies at the head of the spine, evaluation stops and returns the current spine unchanged.

5.3 Primitive Evaluation

Primitive arithmetic operators and the conditional `IfZero` are handled separately before ordinary combinator rewriting.

For an expression headed by a primitive operator such as `Prim Add`, the evaluator recursively evaluates the required operands until they reduce to spines of the form `[Const n]`. Only then is the primitive operation performed:

$$\texttt{Prim Add}\ a\ b \Rightarrow \texttt{Const}(x + y)$$

provided that `evaluate [a] = [Const x]` and `evaluate [b] = [Const y]`.

The same strategy is used for subtraction, multiplication, and division, with division by zero producing a runtime error.

For conditionals, the evaluator first reduces the condition to an integer constant:

$$\texttt{IfZero}\ c\ t_1\ t_2$$

If `evaluate [c]` yields `[Const 0]`, evaluation continues with `t_1`; otherwise it continues with `t_2`. Any non-integer result in these contexts produces a runtime error.

5.4 Evaluation Example

The evaluator was tested on recursive examples written as `.fun` files, including Fibonacci, Lucas, Pell, and Gauss-style summation programs. A representative example is factorial:

```
let fact = fix lambda f. lambda n.
  if iszero n then 1 else n * (f (n - 1))
in fact 4
```

After parsing, the program is represented internally as:

```
Let "fact" (Fix (Lambda "f" (Lambda "n" (IfZero (Var "n") (Const 1)
(Var "n" :* App (Var "f") (Var "n" :- Const 1)))))) (App (Var "fact") (Const 4))
```

Compilation yields the combinator expression:

```
((C :@ I) :@ Const 4) :@ (Y :@ ((B :@ (S :@ ((C :@ IfZero) :@ Const 1))) :@
((B :@ (S :@ Prim Mul)) :@ ((C :@ B) :@ ((C :@ Prim Sub) :@ Const 1)))))
```

Evaluation then unfolds the fixed-point combinator and forces primitive operations only when needed. At a high level, the computation follows the expected recursive expansion:

$$\texttt{fact}\ 4 \Rightarrow 4{*}\texttt{fact}\ 3 \Rightarrow 4{*}(3{*}\texttt{fact}\ 2) \Rightarrow 4{*}(3{*}(2{*}\texttt{fact}\ 1)) \Rightarrow 4{*}(3{*}(2{*}(1{*}\texttt{fact}\ 0))) \Rightarrow 4{*}(3{*}(2{*}(1{*}1))) \Rightarrow 24$$

The evaluator returns:

```
[Const 24]
```

This confirms that recursive unfolding through `Y`, together with primitive forcing inside arithmetic and conditionals, yields the expected result under the previously stated reduction principles.

6 Towards Graph Reduction

6.1 Address-Based Representation

A direct extension of the current compiler is to replace symbolic variables by addresses:

```
Var Addr    type Addr = Int
```

Compilation would then proceed under an explicit environment associating identifiers with addresses.

6.2 Heap Evaluation

Evaluation could then be performed over a heap in which applications and combinators are stored as nodes. Instead of rebuilding $P : @Q$ syntactically after each rewrite, applications would become heap nodes addressed by integers. A machine state would then contain a root address, a stack of addresses, and the heap itself.

Reduction of combinators such as `S`, `B`, and `C` would become a matter of local heap update: new application nodes are allocated and pointers are redirected. Shared subexpressions would remain shared because multiple heap nodes may refer to the same address.

This approach avoids repeated reconstruction of application trees and moves evaluation closer to the graph reduction machines used in practical implementations of functional languages [4].

7 Conclusion

This project implements a small yet complete pipeline for a functional language: source programs are read from `.fun` files, parsed into abstract syntax, compiled into extended combinatory logic, and evaluated by weak head reduction following a call-by-name strategy. λ-abstraction is eliminated through an optimized translation based on Turner's combinators, and execution is performed directly over application spines using local rewrite rules and primitive forcing.

The resulting system shows how functional programs can be executed without environments or substitution at runtime, relying instead on combinator reduction. It also shows how the same ideas lead naturally towards graph-based execution models used in practical implementations of functional languages.

References

[1] Alonzo Church. *The Calculi of Lambda-Conversion*. Princeton University Press, 1941.

[2] Moses Schönfinkel. *Über die Bausteine der mathematischen Logik*. Mathematische Annalen, 92, pp. 305–316, 1924.

[3] David A. Turner. *A New Implementation Technique for Applicative Languages*. Software: Practice and Experience, Vol. 9, 31–49, 1979.

[4] Simon Peyton Jones. *The Implementation of Functional Programming Languages*. Prentice Hall, 1987.

[5] Haskell B. Curry and Robert Feys. *Combinatory Logic, Vol. 1*. North-Holland, 1958.